CSCE 5610 Computer Architecture

Comparative Analysis of FIFO and LRU Page Replacement Algorithms in Virtual Memory

Group 4

Team Members

Ruchitha Sri Sai Valli Thanmayee Potladurthy – 11855553

Adla Sanjay Ramaswamy – 11828288

Dharani Kondapally – 11853256

GitHub Repository:

https://github.com/Dharanikondapally/Group04_CSCE5610_FA26

1. Objective

Our team will develop a Python-based virtual memory simulator to implement and compare two page replacement algorithms: First-In, First-Out (FIFO) and Least Recently Used (LRU). The simulator will process the same page-reference workloads using identical memory-frame configurations and record how each algorithm handles page hits, page faults, and page replacement. The main question we will investigate is: How do FIFO and LRU differ in page-fault and page-hit performance when processing the same page-reference workloads under different memory-frame constraints? Multiple reference strings and frame configurations will be evaluated so that the comparison is based on controlled and measurable results rather than a single test case.

2. Architecture Concept

The central computer architecture concept in this project is virtual memory and memory management. When a requested page is not currently available in the allocated memory frames, a page fault occurs. If all frames are occupied, the system must select an existing page for

replacement. The replacement policy can directly affect how frequently pages need to be loaded into memory.

FIFO and LRU approach this decision differently. FIFO replaces the page that has been in memory the longest, regardless of how recently it was accessed. LRU instead replaces the page that has not been referenced for the longest period of time. By implementing both algorithms under the same memory constraints, the project will demonstrate how different page replacement policies affect page hits, page faults, and memory-frame behavior.

3. Design and Artifact

The main artifact will be a modular Python virtual memory simulator consisting of a common experiment controller, separate FIFO and LRU implementations, metric collection, and result comparison.

The simulator will accept two primary inputs: a page-reference string and the number of available memory frames. The experiment controller will provide identical inputs to FIFO and LRU. Each algorithm will process the reference string sequentially while maintaining its own memory-frame state.

For FIFO, the simulator will maintain the order in which pages enter memory and replace the oldest resident page when replacement is required. For LRU, the simulator will track page usage and replace the resident page whose most recent use occurred furthest in the past.

System Workflow

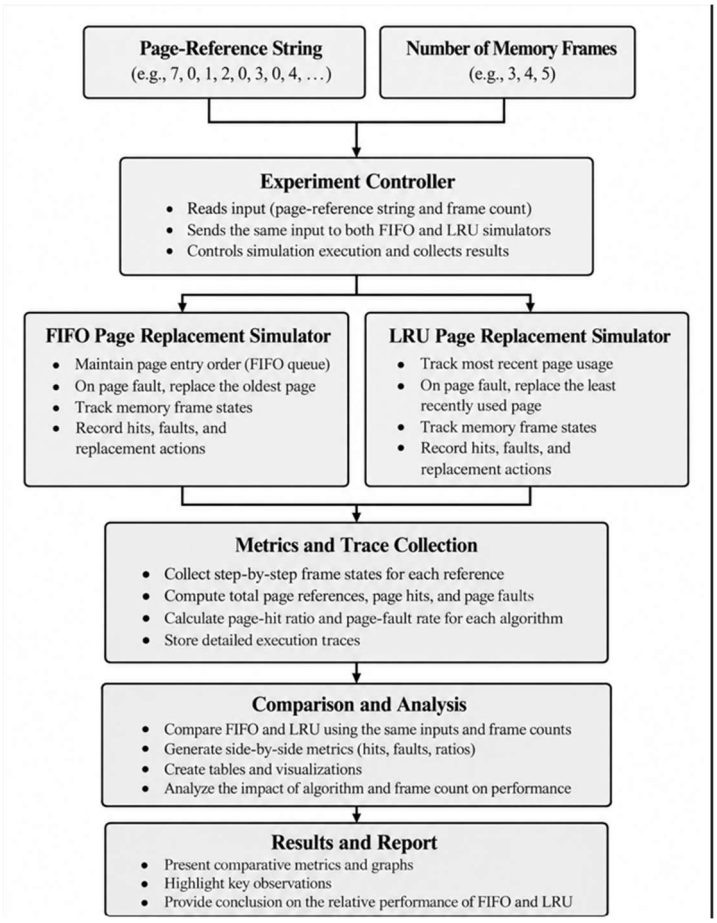


Figure 1. Workflow of the FIFO and LRU Virtual Memory Simulator.

The workflow begins with the page-reference string and memory-frame count. The experiment controller supplies the same inputs to both replacement algorithms. FIFO and LRU independently process the workload and generate execution information. Their results are then passed to a common metric collection stage and compared using the same performance measures.

For each run, the simulator will produce the step-by-step memory-frame states, total page references, page hits, page faults, page-hit ratio, page-fault rate, and final memory-frame state.

4. Comparison

Two configurations will be evaluated under the same experimental conditions.

FIFO: When a page fault occurs and all available frames are occupied, the page that entered memory earliest will be replaced.

LRU: When a page fault occurs and all available frames are occupied, the page whose most recent use occurred furthest in the past will be replaced.

For each experiment, FIFO and LRU will receive exactly the same page-reference string and the same number of memory frames. Multiple fixed reference strings will be tested with different frame configurations, initially including 3, 4, and 5 frames. This controlled setup will allow differences in the measured results to be associated with the replacement policy rather than differences in input conditions.

5. Metric and Test Plan

The primary quantitative metrics will be page faults, page-fault rate, page hits, and page-hit ratio.

Page-Fault Rate = Page Faults / Total Page References

Page-Hit Ratio = Page Hits / Total Page References

The workloads will consist of multiple predefined page-reference strings evaluated under multiple frame counts. Each workload and frame-count combination will be processed by both FIFO and LRU.

Because both algorithms are deterministic for a fixed reference string and frame count, repeating the same unchanged test case should produce the same result. Therefore, the main evaluation will focus on testing multiple workloads and frame configurations rather than performing unnecessary repeated timing trials.

Correctness will first be verified using small reference strings that can be manually traced. The expected frame contents, replacement decisions, page hits, and page faults will be compared with simulator output. The implementation will also verify the following consistency conditions:

Page Hits + Page Faults = Total Page References

Number of Resident Pages ≤ Number of Available Frames

The step-by-step frame trace will provide an additional way to verify that each replacement follows the intended FIFO or LRU policy.

6. Tools and Dependencies

Component	**Tool / Environment**
Programming Language	Python 3
Core Libraries	Python standard library
Development Environment	VS Code or another Python-capable environment
Version Control	Git
Repository	GitHub
Project Management	GitHub Issues and GitHub Projects
Hardware	Standard computer capable of running Python 3
Starter Code	None required
External Simulator	None required

The core implementation will remain self-contained and will not require specialized hardware, privileged operating-system access, or an external dataset. The page-reference workloads will be explicitly defined as simulator inputs, making the experiments easy to reproduce.

7. Schedule and Roles

Timeline	Major Tasks	Owner(s)
Week 1 – Planning and Design	Finalize simulator structure, input/output format, workloads, frame configurations, performance metrics, and correctness test cases.	All Members
Week 2 – Core Implementation	Implement FIFO and LRU, including page-hit/page-fault detection, replacement logic, and frame-state tracking.	Dharani – FIFO; Sanjay – LRU
Week 3 – Integration and Checkpoint	Integrate both algorithms with the common experiment controller, generate execution traces and metrics, and validate the minimum working version.	Ruchitha – Integration; All Members – Testing
Week 4 – Testing and Experimentation	Run controlled experiments using multiple reference strings and frame counts. Verify results against manual test cases and consistency checks.	All Members
Week 5 – Analysis and Results	Compare FIFO and LRU results, organize measurements into tables and visualizations, and analyze differences across workloads and frame configurations.	Ruchitha & Sanjay – Analysis; Dharani – Validation
Week 6 – Finalization	Perform final testing, clean and document the code, finalize repository documentation, organize final results, and prepare the report and presentation materials.	All Members

By the end of **Week 3**, the team will have a working checkpoint version capable of processing identical inputs with FIFO and LRU and producing validated frame traces, page-hit counts, page-fault counts, hit ratios, and fault rates. The remaining weeks will focus on controlled experimentation, analysis, documentation, and final delivery.

8. Risks and Backup Plan

Risk 1 – Incorrect replacement logic or metric calculations: Errors in page tracking, replacement order, or recent-use tracking could produce incorrect results.

Mitigation and Backup: Small, manually traceable reference strings will be used to validate both algorithms before larger experiments are performed. Expected frame transitions and hit/fault counts will be compared with simulator output. Since FIFO and LRU will be implemented as separate modules, either algorithm can also be tested independently if an integration problem occurs.

Risk 2 – Visualization or extended analysis requires more time than expected: Additional plots or presentation features could take time away from validating the core simulator.

Mitigation and Backup: Core implementation, correctness testing, and experimental data collection will be completed before optional visualization work. If additional visualizations cannot be completed reliably, validated comparison tables and frame traces will be used to present the results. This ensures that the primary FIFO-versus-LRU comparison can still be completed within the available project period.